Three-ReLU Pythia-14M: Architecture, Pressure, and Exact-Zero Compute Paths

Thiago Mattar
thiagocmattar@gmail.com

2026-07-13

- **Architecture cost.** Adding the two post-LayerNorm ReLUs produces about 50% exact zeros before QKV and MLP-up, but raises validation loss by 0.1160 versus MLP-ReLU AdamW.
- **Best three-site pressure.** Orthogonal Ricker (OR) raises R_{block} from 19.71% to 28.66% and R_{model} from 5.90% to 8.59%, for +0.0526 validation loss.
- **Scale context.** OR realizes 73.0% of Pythia-14M's 11.76% model-wide architecture ceiling. The same ceiling rises to 54.65% at Pythia-160M and 86.78% at Pythia-12B, so the small absolute 14M result is strongly denominator-limited.
- **L1 behavior.** L1N and OL1 increase attention-input zeros while reducing sparsity at one or both MLP gates; their total compute opportunity is smaller and their quality cost larger.
- **Propagation.** Gate zeros remove products only in the immediately following QKV, W_1, or W_2 multiplication. Dense projections and residual addition do not preserve them.
- **Next step.** Test constrained, cost-weighted thresholds at the three gates, with OR and MLP-only OL1 as references.

1 Additional Related Work

- **Paper and date.** *Sparser, Faster, Lighter Transformer Language Models* [1], **24 March 2026**.
- **Method.** ReLU and mild L1 activation regularization in gated feed-forward blocks, paired with Tile-wise ELLPACK storage and fused CUDA kernels.
- **Result.** At 2B parameters and 40B tokens: 48.8% versus 49.1% mean task accuracy, +20.5% forward throughput, and +21.9% training throughput with more than 99% on FFN/MLP sparsity.
- **Key conclusion.** Validate the feed-forward sparsity-to-kernel chain.

2 Corrected Compute Target

ReLU Strikes Back [2] motivates the relufication pattern. Applied to Pythia's parallel-residual block,

$$U_l = \mathrm{ReLU}(\mathrm{LN}_l^{\mathrm{att}}(H_l)), \qquad \mathrm{Attn}_l = \mathrm{Attention}(U_l),$$
$$M_l = \mathrm{ReLU}(\mathrm{LN}_l^{\mathrm{mlp}}(H_l)), \qquad G_l = \mathrm{ReLU}(M_l W_{1,l}),$$
$$H_{l+1} = H_l + \mathrm{Attn}_l + G_l W_{2,l}.$$

- **Three-ReLU** means ReLU at `attention_inputs`, `mlp_inputs`, and `mlp_hiddens` in all six blocks. The final model LayerNorm is unchanged.
- The three direct targets are immediately before QKV, MLP-up W_1, and MLP-down W_2.
- `attention_outputs` and `residual_streams` are too late: their producer matmuls have already executed, and the next dense projection or LayerNorm does not preserve their zero mask.

Pythia-14M has $L = 6$ blocks, hidden width $d = 128$, MLP width $4d = 512$, sequence length $T = 2048$, and vocabulary size $V = 50{,}304$. Let $z_a, z_m, z_h \in [0, 1]$ be exact-zero fractions at the attention input, MLP input, and MLP hidden. Define

$$C_{\mathrm{target}}(z) = 3d^2 z_a + 4d^2 z_m + 4d^2 z_h,$$
$$C_{\mathrm{block}} = 12d^2 + d(T+1),$$
$$C_{\mathrm{model}} = LC_{\mathrm{block}} + dV,$$
$$R_{\mathrm{block}}(z) = \frac{C_{\mathrm{target}}(z)}{C_{\mathrm{block}}}, \qquad R_{\mathrm{model}}(z) = \frac{LC_{\mathrm{target}}(z)}{C_{\mathrm{model}}}.$$

The $d(T+1)$ term is the combined valid-causal QK-score and PV-mix cost, averaged per token. R_{block} uses one complete transformer block as its denominator; R_{model} uses all six blocks plus the final $d \times V$ language-model projection. Both are potentially avoidable scalar-product fractions, not measured speedups.

For this model and sequence length:

- If all three gate inputs were zero, $C_{\mathrm{target}}(1,1,1) = 11d^2 = 180{,}224$ products per token per block.
- $C_{\mathrm{block}} = 458{,}880$ and $C_{\mathrm{model}} = 9{,}192{,}192$ products per token.
- Therefore $R_{\mathrm{block}}(1,1,1) = 39.2748\%$ and $R_{\mathrm{model}}(1,1,1) = 11.7637\%$. These are architecture ceilings; measured values use the observed z_a, z_m, z_h in Table 3.

Table 1: Scalar-product accounting for one Pythia-14M block at $T = 2048$. Future causal-mask entries are excluded. "Target gate" names the upstream activation whose exact zeros can remove products without changing the dense result.

Operation	Row-vector operation	Products/token	Share of full block	Target gate
QKV projection	$[T \times d][d \times 3d] \to [T \times 3d]$	49,152	10.711%	Attention input
QK scores	valid $QK^\top$	131,136	28.577%	Dense projected Q, K
PV mix	valid PV	131,136	28.577%	Dense P, V
Output projection	$[T \times d][d \times d] \to [T \times d]$	16,384	3.570%	Dense context
MLP up (W_1)	$[T \times d][d \times 4d] \to [T \times 4d]$	65,536	14.282%	MLP input
MLP down (W_2)	$[T \times 4d][4d \times d] \to [T \times d]$	65,536	14.282%	MLP hidden
Directly gated total	QKV $+W_1 + W_2$	180,224	39.275%	Three ReLU gates
One complete block	All operations above	458,880	100.000%	—

The ceiling falls from 39.27% block-local to 11.76% model-wide because the second denominator includes the dense, ungated final language-model projection, which dominates the matmul count in this small model.

For site s and layer l, exact-zero percentage is

$$z_{s,l} = 100 \, \frac{\sum_{b,t,j} \mathbf{1}[A_{btj}^{(s,l)} = 0]}{\sum_{b,t,j} 1}.$$

The comparison is direct equality to floating-point zero on the captured forward tensor; it is not a histogram-bin estimate and does not count merely small values. Counters cover all 692,224 tokens in the deterministic validation cache, all coordinates, and all six layers. Reported site aggregates pool their integer hit and element counts.

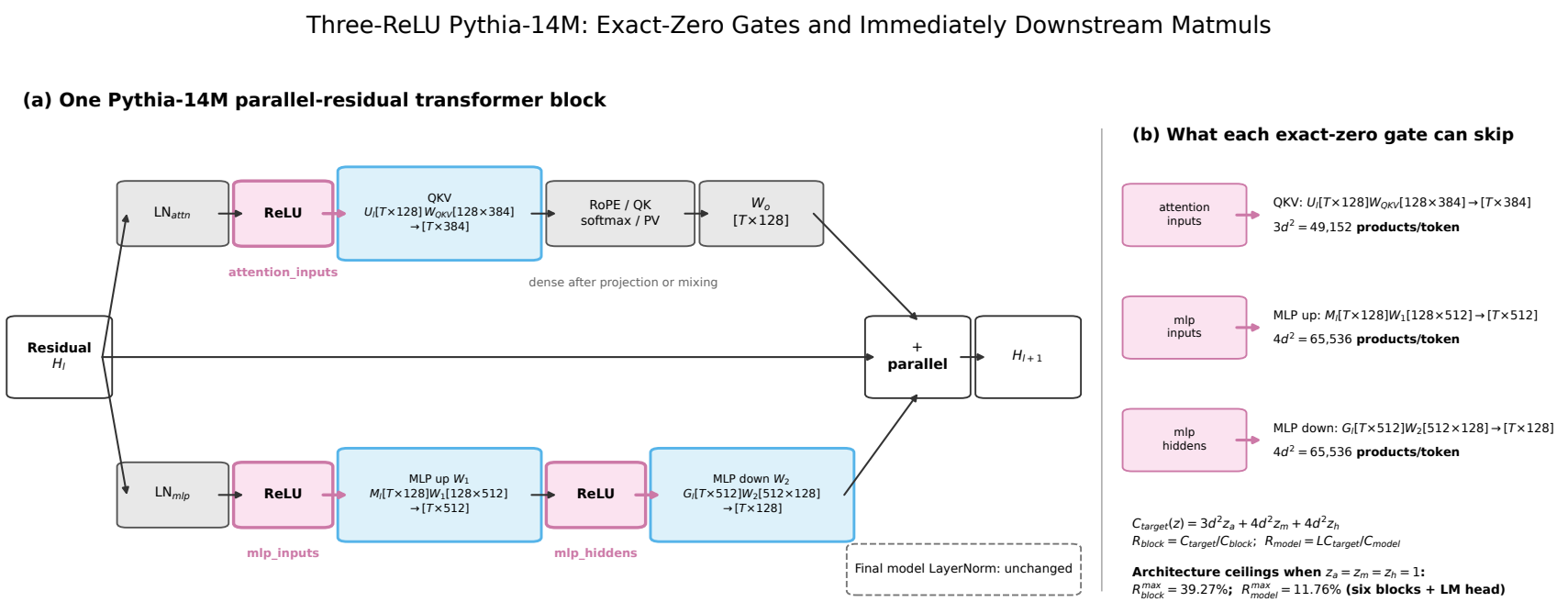

Figure 1: The Pythia parallel-residual block, explicit tensor and matrix dimensions for QKV, W_1, and W_2, and the products controlled by each ReLU gate. The displayed R_{block} and R_{model} values are maxima at $z_a = z_m = z_h = 1$, not observed speedups.

Pythia-family scaling. At the all-zero gate ceiling, the same definitions simplify to

$$R^{\max}_{\text{block}} = \frac{11d}{12d+T+1}, \qquad R^{\max}_{\text{model}} = \frac{11Ld}{L(12d+T+1)+V}.$$

Substituting the official L, d, and padded V values for the Pythia family [3] gives Figure 2.

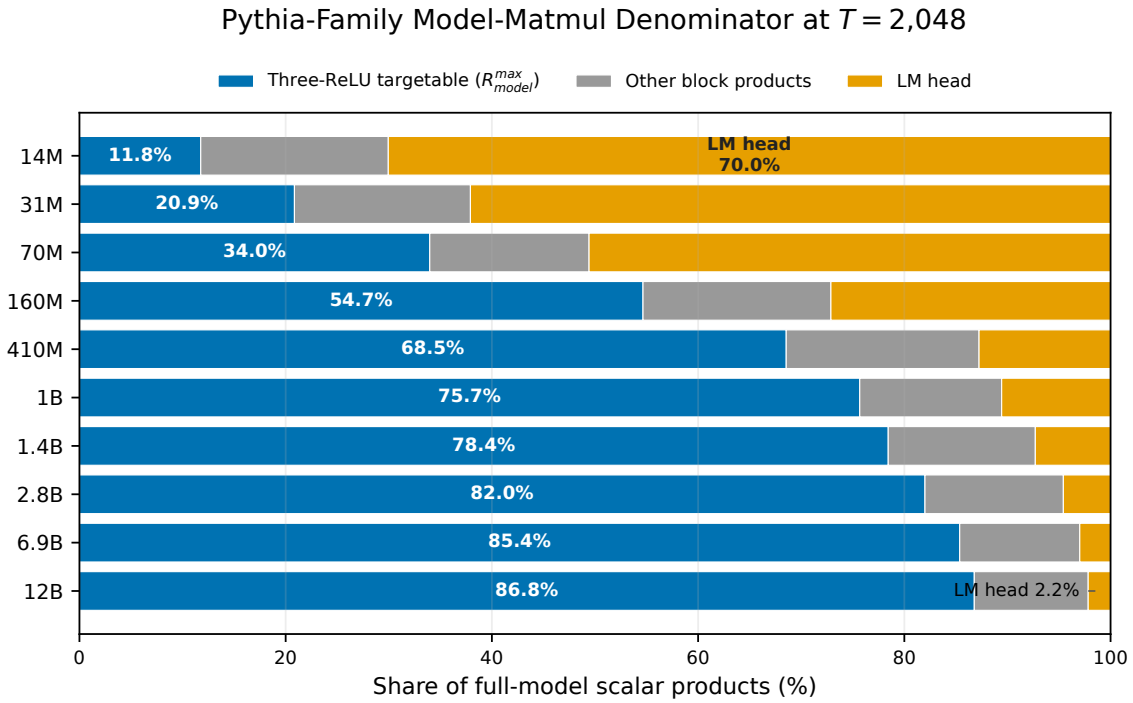

Figure 2: Pythia-family decomposition of the full-model scalar-product denominator at $T = 2048$. Blue is the maximum Three-ReLU-targetable share $R^{\max}_{\text{model}}$; gray is the untargeted W_o plus valid-causal QK and PV work; orange is the LM head. Every bar assumes $z_a = z_m = z_h = 1$, so these are architecture ceilings rather than observed sparsity or speedups.

- **Scale effect.** $R^{\max}_{\text{model}}$ rises from 11.76% at 14M to 33.97% at 70M, 54.65% at 160M, 68.55% at 410M, and 86.78% at 12B. The LM-head share simultaneously falls from 70.05% to 2.17%.
- **Why.** Targeted projection work grows as d^2, while the attention core grows as dT; additional blocks also amortize the single dV vocabulary projection.
- **Ceiling utilization.** Define $U(z) = R_{\text{model}}(z)/R^{\max}_{\text{model}} = (3z_a+4z_m+4z_h)/11$. Three-ReLU OR reaches $U = 73.0\%$ on 14M: its 8.59% absolute value is low largely because 14M has a low ceiling, not because OR creates few gate zeros.
- **Scope.** This is full-sequence, full-logit accounting at $T = 2048$. It does not predict larger-model zero rates; those require training and measurement, and decoding with a KV cache has a different denominator.

3 Experiment and Nomenclature

Every training run starts from random initialization and consumes one MiniPile token-cache pass: 22,762 optimizer steps and 1,491,730,432 tokens. Validation loss is evaluated on 85 deterministic batches (692,224 tokens). Pressure in the Three-ReLU runs is applied only at the three ReLU outputs.

Table 2: Architectures, method settings, and one-pass endpoints. OR is orthogonal Ricker, L1N is naive L1, and OL1 is orthogonal L1. Parameters lists only nondefault settings.

Config	Architecture	Name	Parameters	Validation loss	Time (h)	Tokens/s
50	GELU	AdamW	—	4.8317	3.010	137,648
77	MLP-ReLU	AdamW	—	4.8404	2.897	143,021
81	MLP-ReLU	OL1	$w = 5$, budget $= 0.5$; MLP hidden only	4.7981	3.788	109,379
98	Three-ReLU	AdamW	—	4.9564	3.449	120,139
103	Three-ReLU	OR	$w = 1$, $c = \sigma = 0.05$, budget $= 0.5$	5.0091	4.404	94,086
104	Three-ReLU	L1N	$w = 5$	5.1882	5.597	74,029
99	Three-ReLU	OL1	$w = 5$, budget $= 0.5$	5.1706	4.318	95,971

4 Architecture and Pressure Effects

Table 3: Exact-zero outcomes for MLP-ReLU AdamW and the matched Three-ReLU methods. Site percentages pool all layers and all 692,224 validation tokens.

Config	Method	z_a	z_m	z_h	R_{block}	R_{model}
77	MLP-ReLU AdamW	0.00%	0.00%	52.06%	7.44%	2.23%
98	Three-ReLU AdamW	49.54%	50.14%	50.72%	19.71%	5.90%
103	Three-ReLU OR	72.74%	71.00%	75.15%	28.66%	8.59%
104	Three-ReLU L1N	80.38%	44.50%	54.19%	22.70%	6.80%
99	Three-ReLU OL1	83.07%	39.25%	48.87%	21.48%	6.43%

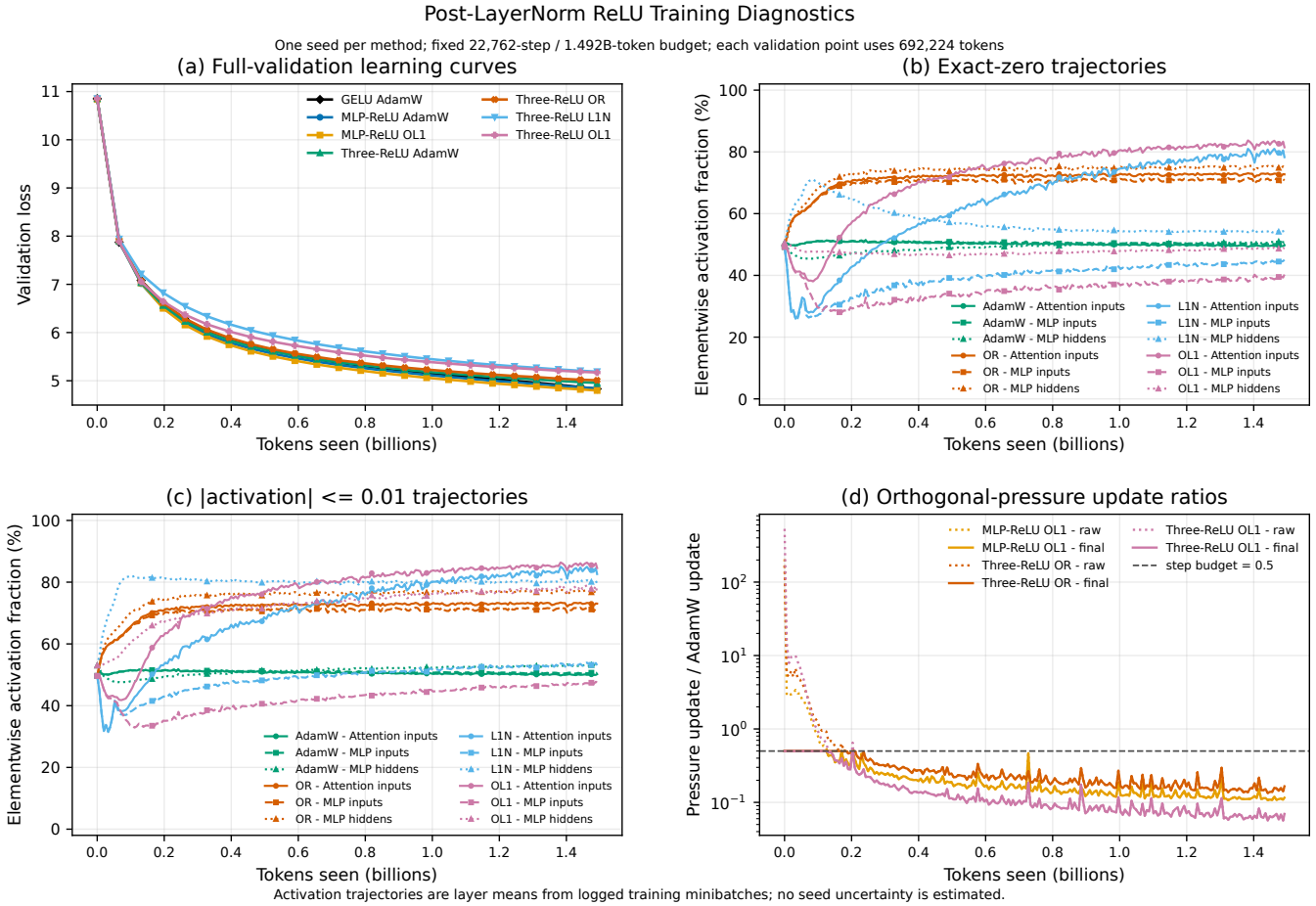

Figure 3: Learning and pressure diagnostics for the complete control ladder. Colors identify methods consistently throughout the report. Validation loss is separated from exact-zero and near-zero trajectories; orthogonal-method panels show the bounded Adam-step correction rather than conflating it with naive auxiliary-loss training.

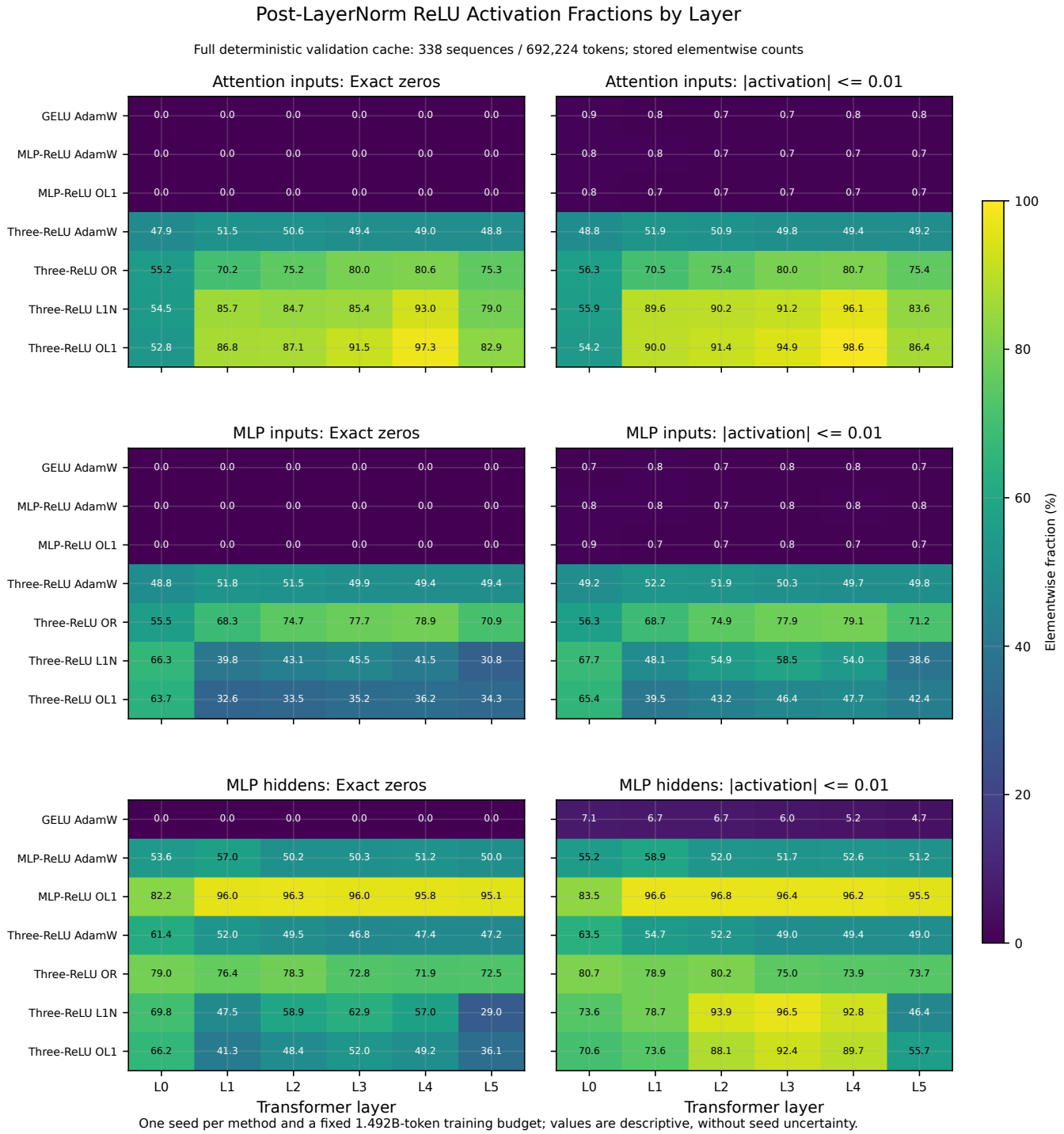

Figure 4: Full-validation exact-zero and $|a| \leq 0.01$ mass by activation site and layer. Direct counters, rather than density bins, determine the percentages. The comparison separates native ReLU gates from pressure-induced changes and exposes layer-specific operating points hidden by a global average.

5 How Exact Zeros Propagate

The propagation path answers precisely where skip opportunities exist.

Attention branch. A zero coordinate in U_l multiplies an entire corresponding row of each Q, K, and V weight matrix, so it creates $3d$ exactly zero input–weight products. It does *not* normally create a zero Q, K, or V output coordinate: each output is a sum over the other input coordinates and a bias. QK scores are therefore dense sums;

softmax is strictly positive on valid causal positions; the PV mix is dense; and the attention output projection sums dense context coordinates. Residual addition then combines a dense attention output with H_l and the MLP output. The original gate zeros have saved QKV products but have not survived as output zeros.

MLP branch. A zero in M_l creates $4d$ zero products in MLP-up. The up-projection output is again a dense sum. The following hidden ReLU creates a *new* exact-zero mask G_l, whose zeros create d zero products each in MLP-down. The down-projection output is dense, and residual addition is dense. Thus MLP-input and MLP-hidden zeros are two distinct skip interfaces, not one mask propagating through both matrices.

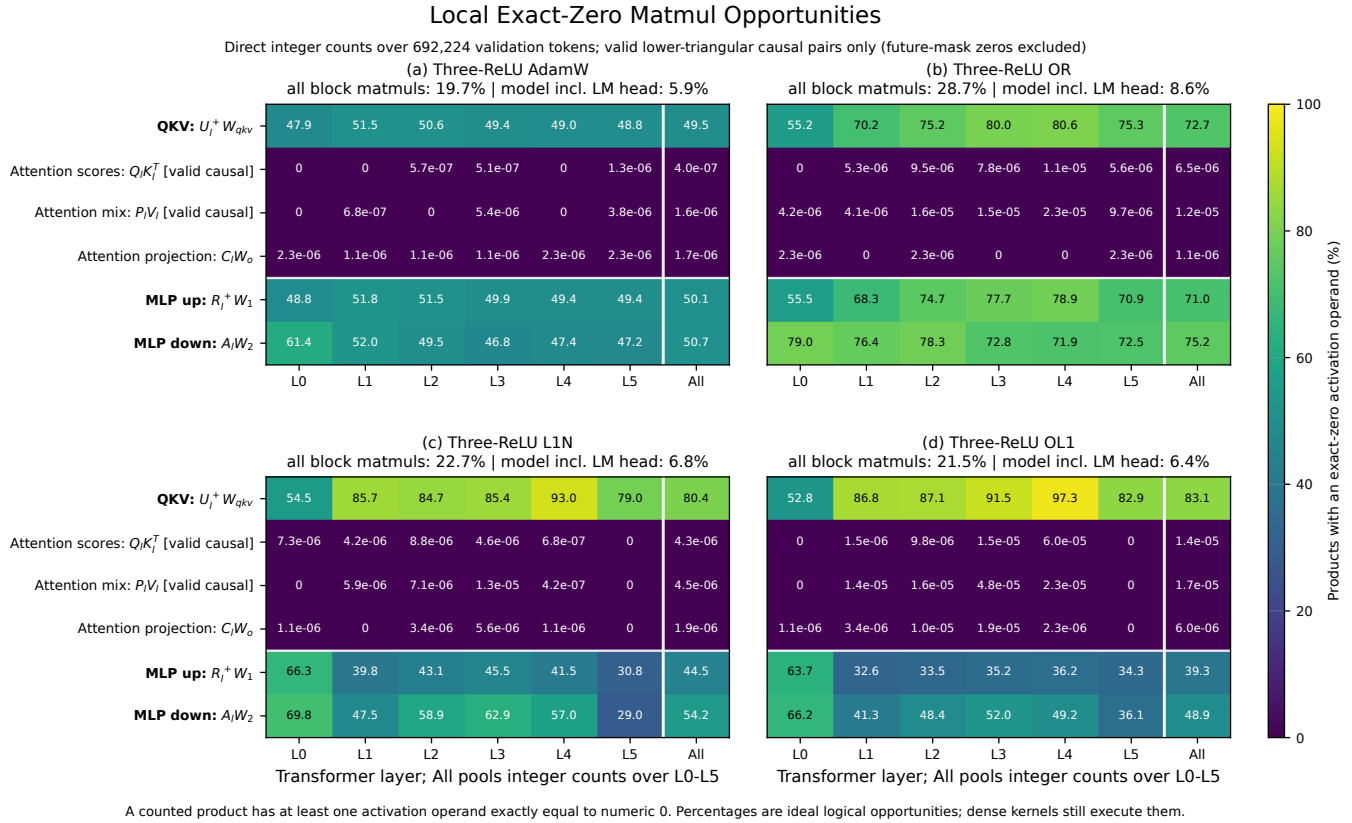

Figure 5: Logical exact-zero product percentages for QKV, valid-pair QK, valid-pair PV, attention output, MLP-up, and MLP-down matmuls. A linear-layer product is counted as avoidable when its activation input is exactly zero; an attention-core product is zero when either multiplicand is exactly zero. Future causal-mask pairs are excluded. QKV, MLP-up, and MLP-down inherit large opportunities from their immediately upstream gates, whereas subsequent dense sums leave essentially none.

Figure 6: Exact-zero activation propagation for Three-ReLU AdamW, OR, L1N, and OL1. Each percentage is computed across all 692,224 validation tokens, all coordinates at that named interface, and the indicated layer. The large page is intentional: it preserves per-layer labels and makes the immediate densification after QKV, MLP-up, MLP-down, and residual addition visible.

Across the four Three-ReLU checkpoints, dense downstream activation boundaries contain only about $10^{-7}\%$ to $1.7 \times 10^{-5}\%$ exact-zero mass, and valid attention probabilities contain none. The heatmaps therefore reject a "zeros cascade through the layer" interpretation. Sparsity is local and consumable only at the matmul immediately following each gate; the hidden MLP ReLU creates a second independent mask.

6 Activation, Weight, and LayerNorm Distributions

Exact-zero rates do not describe the nonzero tail. Three-site pressure also changes activation scale and can create a reservoir of small positive values. For example, relative to Three-ReLU AdamW, OL1 reduces pooled activation RMS from 0.723 to 0.172 at attention inputs, 0.705 to 0.148 at MLP inputs, and 0.115 to 0.020 at MLP hiddens. This distinction matters: a small positive activation is not free in an exact sparse kernel, and clipping can show that some near-zero mass is functionally important.

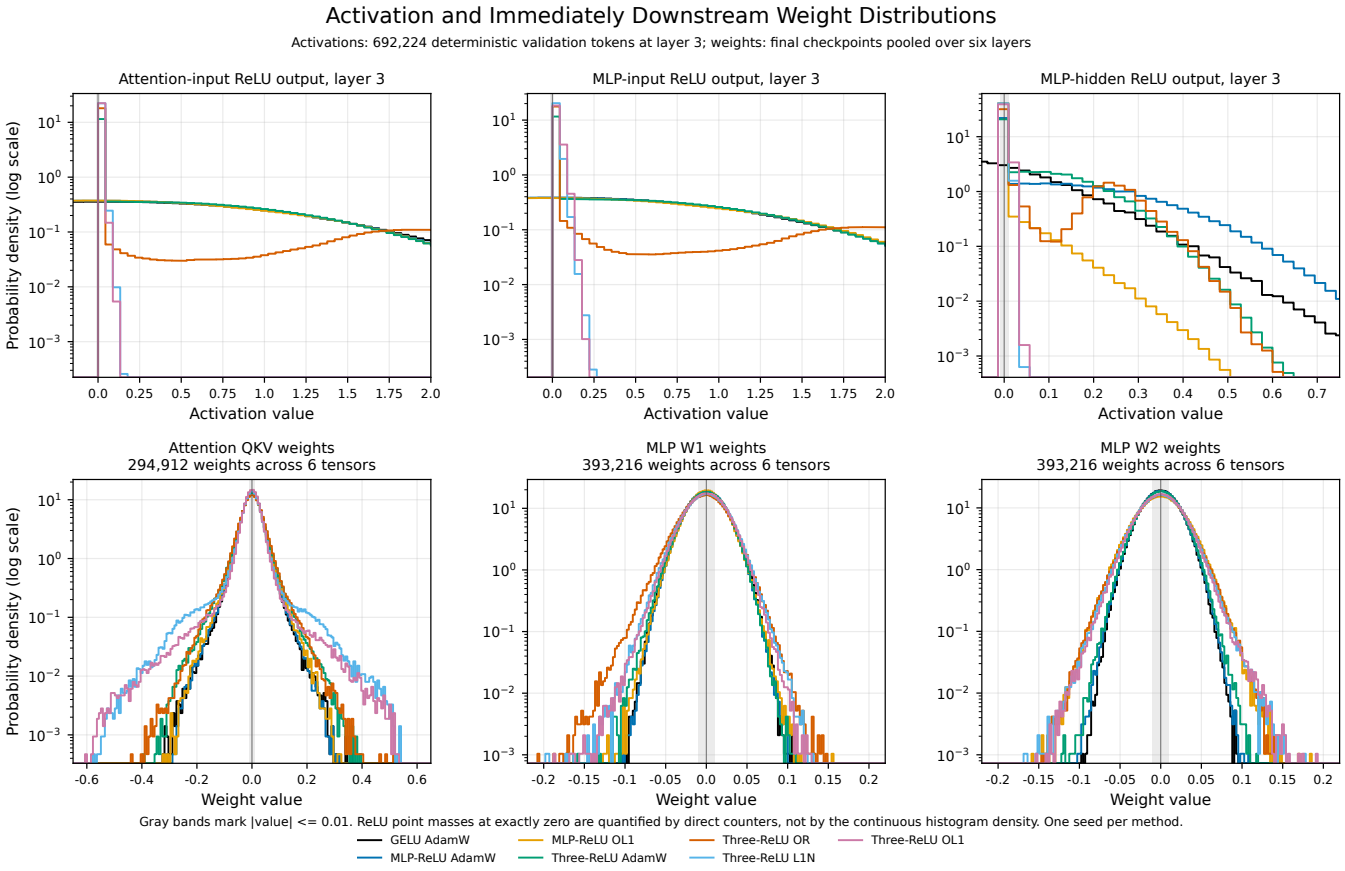

Figure 7: Activation and immediately downstream weight densities for the control ladder, using shared method colors. Pairing each activation with the matrix that consumes it distinguishes hard gates, small-positive reservoirs, and weight adaptation. Exact-zero point masses are reported separately from direct counters rather than inferred from continuous density estimates.

The downstream weights adapt but do not become sparse. In the original matched comparison, OL1 versus Three-ReLU AdamW increases QKV weight RMS from 0.0433 to 0.0493, MLP-up from 0.0219 to 0.0235, and MLP-down from 0.0218 to 0.0251. This broadening is associated with smaller activation scales, but the observational density plot cannot establish that weights causally compensate for sparse activations.

The branch LayerNorm is

$$\mathrm{LN}(x) = \gamma \odot \frac{x - \mu(x)}{\sqrt{\mathrm{var}(x) + \epsilon}} + \beta, \qquad A = \mathrm{ReLU}(\mathrm{LN}(x)).$$

Here γ controls normalized scale and β moves coordinates across the zero gate. OL1 changes both: relative to config 98, mean attention-LayerNorm γ moves from 1.017 to 0.870 and mean β from $+0.017$ to -0.135; mean MLP-LayerNorm

γ moves from 1.003 to 0.918 and mean β from $+0.003$ to -0.089. These parameter shifts explain how a fixed zero threshold can acquire very different site- and layer-specific gate rates.

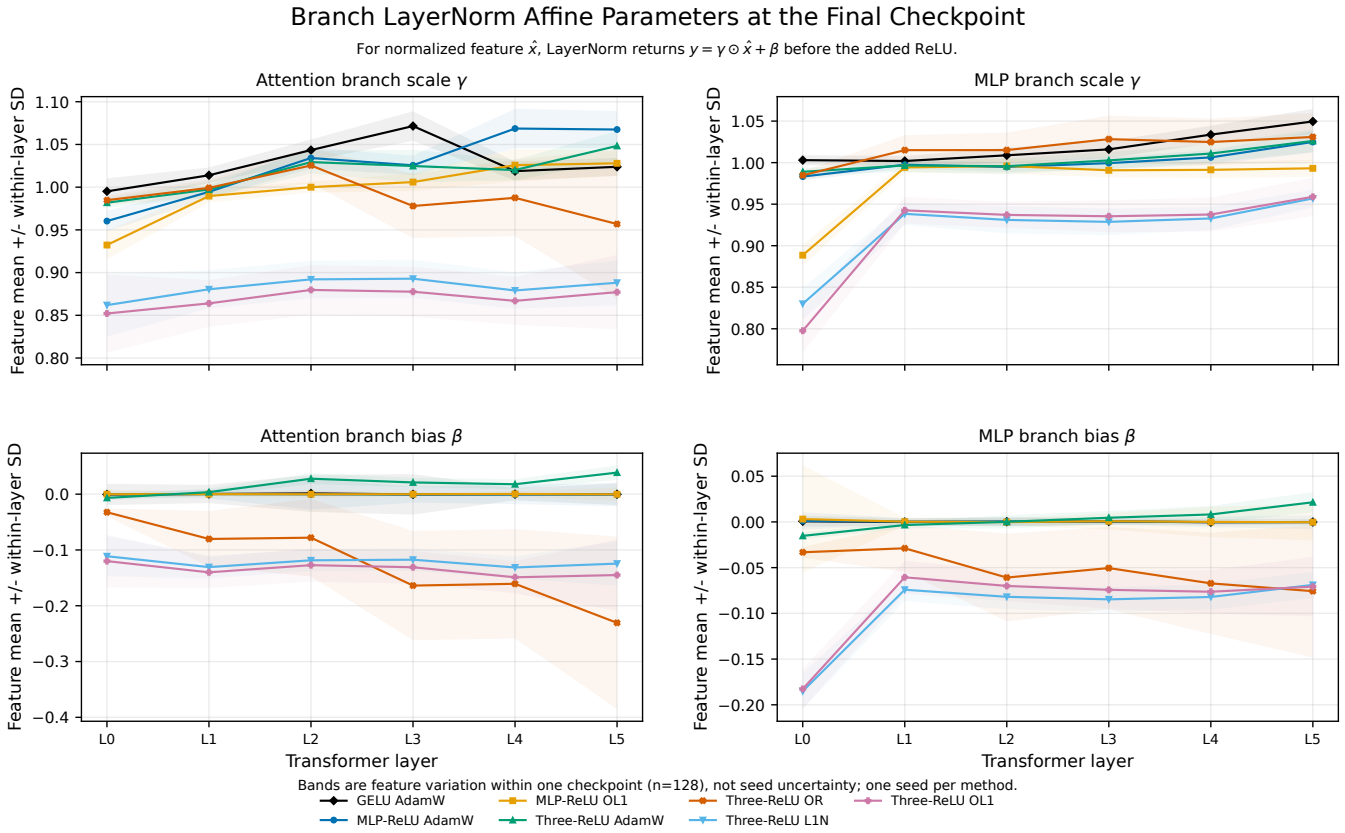

Figure 8: Attention-branch and MLP-branch LayerNorm affine parameters, separated from downstream matrix weights. Negative bias shifts more normalized coordinates below the ReLU threshold; scale changes alter the width of the surviving distribution. Values are associations at the final checkpoint, not a causal decomposition.

A natural architectural follow-up is a learnable gate

$$A_l^{(s)} = \mathrm{ReLU}\Big(\mathrm{LN}_l^{(s)}(H_l) - \tau_l^{(s)}\Big),$$

with a global compute budget allocating thresholds $\tau_l^{(s)}$ across layers and sites. A free per-coordinate τ is algebraically redundant with LayerNorm bias, so the useful version should be constrained—for example one scalar per layer/site, monotone or budget-regularized—and evaluated by the compute it actually controls. This makes the gate policy explicit instead of asking unconstrained LayerNorm biases to discover it indirectly.

7 Post-hoc Clipping: Robustness and the Compute Frontier

Clipping sweeps use the complete validation cache and replace values with zero when $|a| \leq t$. Threshold zero is the unmodified checkpoint. Site-specific sweeps isolate which near-zero reservoirs are safe; joint sweeps clip all three gates and convert resulting exact zeros to the model-wide denominator above.

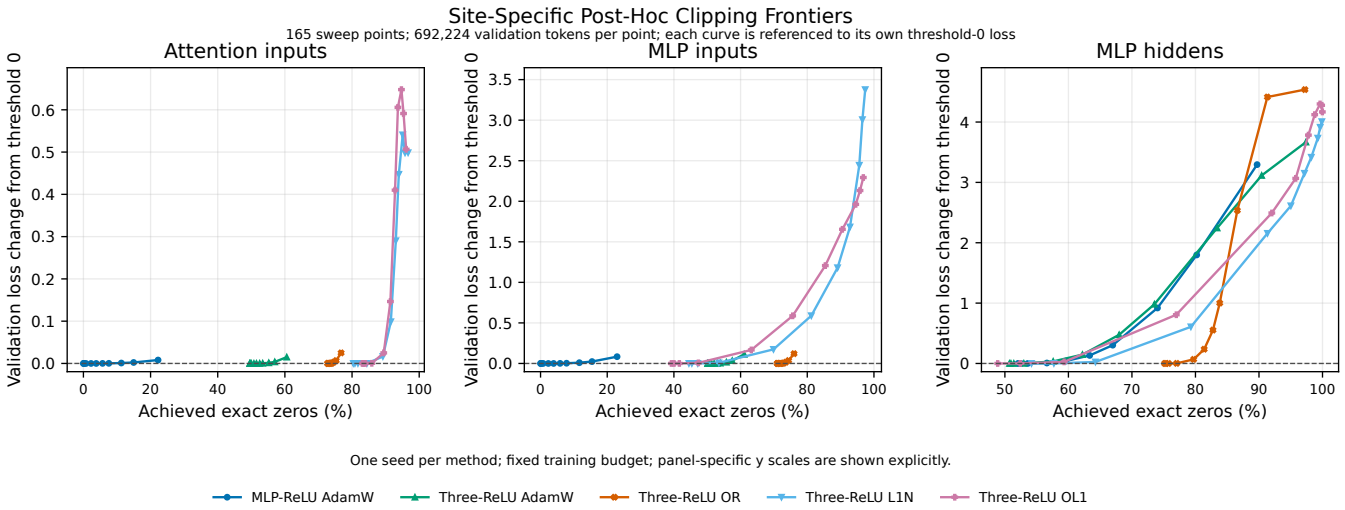

Figure 9: Site-specific clipping frontiers. Validation-loss change is measured from each checkpoint's own threshold-zero loss, separating clipping fragility from training quality. The panels show that equal near-zero mass does not imply equal removable computation across attention input, MLP input, and MLP hidden.

The earlier OL1 result illustrates the point sharply: at $t = 0.01$, attention-input clipping adds only 0.00034 validation loss and MLP-input clipping adds 0.00719, whereas MLP-hidden clipping adds 0.8074. Its large MLP-hidden $|a| \leq 0.01$ reservoir is therefore not a safe exact-zero operating point.

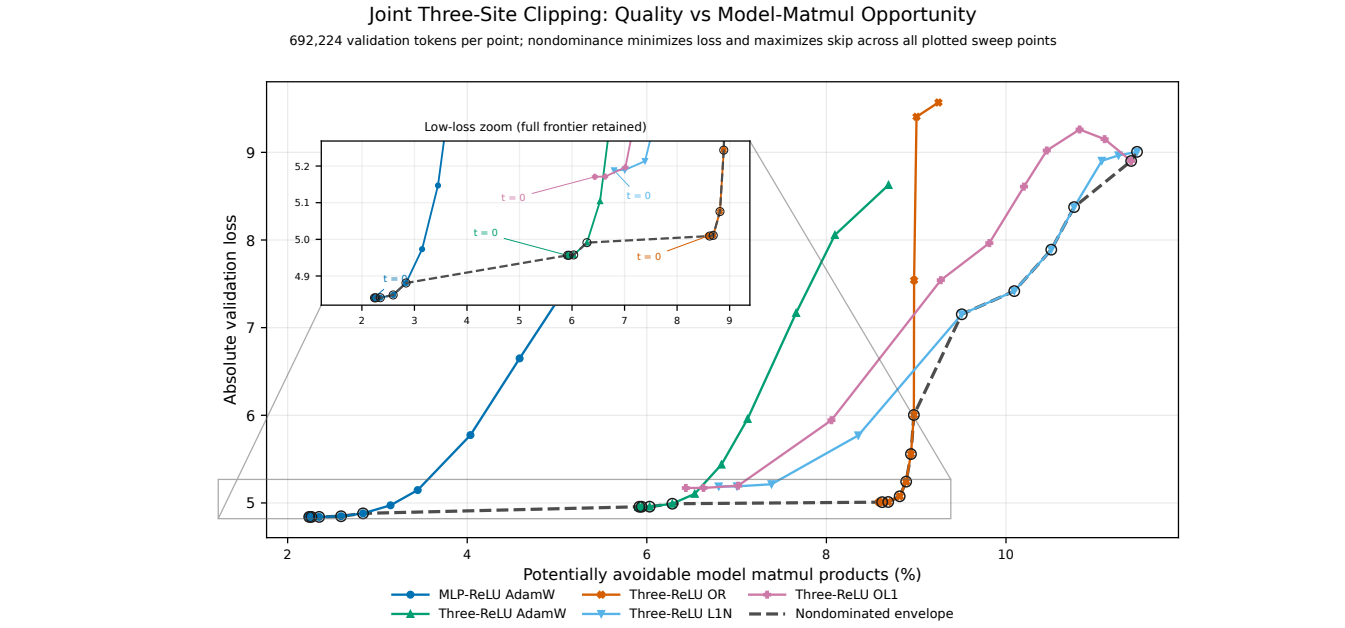

Figure 10: Joint three-site clipping frontier against potentially avoidable *model-wide* matmul products. The denominator is six complete transformer blocks plus the unprunable final language-model projection at $T = 2048$; therefore $R_{\text{model}}(1, 1, 1) = 11.7637\%$ is the architecture ceiling. These are logical products, not dense-PyTorch speedups.

OR owns the useful pressured frontier in this test. Its unmodified checkpoint starts at 8.5856% potentially avoidable model matmuls and loss 5.0091. Clipping at $t = 0.003$ reaches 8.6208% with a numerically 0.00009 lower loss; $t = 0.01$ reaches 8.6882% for only +0.00179 loss. L1N starts at 6.8006% and needs $t = 0.01$ to reach 8.3551%, at a +0.5823 loss cost. The frontier therefore confirms the direct counters: balanced sparsity at all three compute-facing sites matters more than maximizing a single site's zero rate.

The head-inclusive percentage is deliberately smaller than the 28.66% OR block number. The final vocabulary projection dominates this small model's full-logit matmul count and has no ReLU gate. Both denominators are useful: R_{block} isolates the mechanism inside a transformer block, while R_{model} prevents that local opportunity from being presented as whole-model savings.

8 Conclusions and Recommended Next Step

This test answers five questions. First, post-LayerNorm ReLUs do create genuine, directly consumable zeros before QKV and MLP-up, but the tested Three-ReLU architecture costs 0.1160 validation loss relative to MLP-ReLU AdamW. Second, pressure is method-dependent: OR produces a balanced rise across all three gates and is the only positive three-site candidate, while L1N and OL1 reveal adverse cross-site redistribution. Third, ReLU zeros do not propagate as zeros through ordinary dense layers; each gate only exposes products in its immediately downstream matmul, and the MLP hidden ReLU creates a new mask. Fourth, exact-zero, near-zero, weight, and LayerNorm distributions together show how the model changes its gate operating points, but they do not by themselves establish a causal compensation mechanism. Fifth, the low absolute R_{model} at 14M is strongly architecture-limited: OR already uses 73.0% of its ceiling, and the ceiling rises sharply with Pythia scale.

The research agenda should now move from local sparsity maximization to a global, hardware-aware activation budget:

1. **Separate the architecture cost.** Train attention-input-only and MLP-input-only post-LayerNorm ReLU ablations against MLP-ReLU AdamW. Keep the final LayerNorm untouched.
2. **Carry OR and MLP-only OL1 as references.** OR is the three-site frontier candidate; config 81 is the quality-preserving narrow-scope reference. Do not carry L1N or three-site OL1 at the present weights unchanged.
3. **Learn where to spend zeros.** Replace equal site averaging with the model-matmul cost weights and constrained learned thresholds. Optimize one global budget over layer–site gates, not three independent sparsity percentages.
4. **Test redistribution causally.** Intervene on one gate at a time—freeze, clamp, or remove its pressure—and measure changes at downstream and parallel sites. Use "cross-site coupling" for observational results and reserve "compensation" for effects established by these interventions.
5. **Match the representation to execution.** After the gate policy is stable across seeds, test token-, channel-, or block-structured masks and a zero-aware kernel. Compare logical product reduction, kernel latency, memory traffic, and energy rather than treating zeros as measured speed.
6. **Scale only the surviving design.** Repeat the architecture ablation and OR/threshold candidate across seeds at 14M, then measure them anew at 70M and 160M, whose ceilings are 33.97% and 54.65%. Do not project the 14M zero rates upward. The present one-seed, one-pass result is mechanism and planning evidence, not an uncertainty-qualified final estimate.

The central recommendation is therefore specific: **build the next experiment around constrained, cost-weighted ReLU thresholds before QKV, MLP-up, and MLP-down, with OR as the pressured comparator and exact-zero propagation as a required diagnostic.**

References

[1] Edoardo Cetin, Stefano Peluchetti, Emilio Castillo, Akira Naruse, Mana Murakami, and Llion Jones. *Sparser, Faster, Lighter Transformer Language Models.* arXiv:2603.23198, first posted 24 March 2026. https://arxiv.org/abs/2603.23198

[2] Iman Mirzadeh, Keivan Alizadeh, Sachin Mehta, Carlo C. Del Mundo, Oncel Tuzel, Golnoosh Samei, Mohammad Rastegari, and Mehrdad Farajtabar. *ReLU Strikes Back: Exploiting Activation Sparsity in Large Language Models.* International Conference on Learning Representations, 2024; arXiv:2310.04564. https://arxiv.org/abs/2310.04564

[3] EleutherAI. *Pythia: Interpreting Transformers Across Time and Scale—Official Model Table and Released Configurations.* https://github.com/EleutherAI/pythia#models; model configurations under https://huggingface.co/EleutherAI. Accessed 13 July 2026.